NestJS cơ bản cho người mới bắt đầu (dùng SQL Server)

Nov 6, 2026 · @Quinn

Lời nói đầu

Tài liệu này dành cho người **mới bắt đầu học NestJS**. Mục tiêu sau 10 bài: tự xây được một API quản lý công việc có đăng ký, đăng nhập, lưu dữ liệu trong **Microsoft SQL Server**.

NestJS là gì?

NestJS là **framework để viết backend (API) bằng Node.js và TypeScript**. Nếu so với Express thuần thì:

- **Express** giống một bộ dụng cụ rời: bạn tự quyết cách sắp xếp code.
- **NestJS** giống một ngôi nhà đã có sẵn khung: có quy định rõ đặt code ở đâu (module, controller, service). Vì vậy dự án nhiều người làm vẫn gọn gàng, người mới vào đọc code cũng nhanh.

Bên dưới, NestJS vẫn dùng Express để chạy, nên kiến thức Node.js bạn đã có không bị phí.

Bạn cần biết gì trước?

- JavaScript cơ bản: biến, hàm, mảng, object, `async/await`.
- Biết API là gì, các phương thức HTTP (GET, POST, PATCH, DELETE).
- Chưa biết TypeScript và SQL cũng được, Bài 2 và Bài 6 sẽ dạy phần cần thiết.

Danh sách bài học

Bài	Nội dung	Sau bài học bạn làm được
1	Cài đặt môi trường	Chạy được Node.js, Nest CLI, SQL Server
2	TypeScript tối thiểu	Đọc hiểu code NestJS
3	Dự án đầu tiên, controller	Tạo API trả dữ liệu
4	Service, module, dependency injection	Tổ chức code đúng cách NestJS
5	DTO và kiểm tra dữ liệu	Chặn dữ liệu sai gửi lên

Bài	Nội dung	Sau bài học bạn làm được
6	SQL Server cơ bản	Tạo bảng, viết câu lệnh T-SQL
7	Kết nối NestJS với SQL Server	API đọc/ghi dữ liệu thật
8	Quan hệ bảng và xử lý lỗi	Liên kết người dùng với công việc
9	Đăng ký, đăng nhập JWT	Bảo vệ API
10	Swagger và unit test	Thử API trên trình duyệt, viết test

Thời gian gợi ý: khoảng **3–4 tuần**, mỗi ngày 1–2 giờ.

Cách học hiệu quả

1. Đọc hết bài một lượt để hiểu ý chính.
2. **Gõ lại code**, đừng copy. Gõ sai rồi tự sửa là cách nhớ lâu nhất.
3. Chạy thử, xem kết quả, cố tình làm sai một chỗ để xem lỗi trông thế nào.
4. Làm phần "Tự luyện" cuối mỗi bài.

Về phiên bản và độ tin cậy của code

Tài liệu viết theo **NestJS 12** (bản mới nhất lúc soạn, tháng 11/2026), dùng **TypeORM** để làm việc với SQL Server.

Toàn bộ code NestJS trong tài liệu đã được **biên dịch và chạy unit test** thành công. Tuy nhiên, môi trường soạn tài liệu không cài được SQL Server, nên phần kết nối và truy vấn trên SQL Server thật **chưa được chạy thử**. Phần này viết theo tài liệu của TypeORM và SQL Server; nếu bạn gặp khác biệt khi chạy, hãy đối chiếu thông báo lỗi với tài liệu chính thức.

Khi xem video hay bài viết cũ trên mạng, bạn sẽ thấy nhiều khác biệt nhỏ: code cũ không ghi đuôi `.js` khi import, dùng Jest thay vì Vitest để test. Đó là do phiên bản cũ, ý tưởng vẫn giống nhau.

Bài 1 — Cài đặt môi trường

Cuối bài này máy bạn sẽ có ba thứ:

- **Node.js:** chạy JavaScript/TypeScript.
- **Nest CLI:** công cụ tạo dự án.

- **SQL Server:** cơ sở dữ liệu.

Ngoài ra cần một công cụ để xem dữ liệu trong SQL Server.

1.1 Cài Node.js

1. Vào trang **nodejs.org**, tải bản có nhãn **LTS** (bản ổn định, được hỗ trợ lâu dài).
2. Cài đặt như phần mềm bình thường.
3. Mở Terminal (Windows: PowerShell hoặc Command Prompt) và kiểm tra:

```
node -v   # ví dụ: v24.x.x
npm -v
```

Node.js phải từ **bản 20 trở lên** thì NestJS 12 mới chạy được.

1.2 Cài trình soạn code

Dùng **Visual Studio Code** (miễn phí). Nên cài thêm các extension:

- **SQL Server (mssql)** của Microsoft: kết nối và chạy câu lệnh SQL ngay trong VS Code.
- **Prettier:** tự định dạng code.

1.3 Cài SQL Server

Chọn **một** trong hai cách.

Cách A — Cài trực tiếp trên Windows (dễ nhất cho người mới dùng Windows)

1. Tải **SQL Server Developer Edition** từ trang SQL Server của Microsoft. Bản này miễn phí, đầy đủ tính năng, nhưng chỉ dùng cho học tập và phát triển. Hoặc tải bản **Express**: miễn phí, có giới hạn dung lượng, được dùng cả cho ứng dụng nhỏ thật.
2. Khi cài, ở bước *Authentication Mode*, chọn **Mixed Mode (SQL Server authentication and Windows authentication)** và đặt mật khẩu cho tài khoản `sa`.
3. Cài thêm **SQL Server Management Studio (SSMS)**: công cụ quản lý SQL Server chính thức trên Windows.

Hai bước bắt buộc sau khi cài trên Windows. Nếu bỏ qua, NestJS sẽ **không kết nối được**:

- **Bật TCP/IP:** mở *SQL Server Configuration Manager* → *SQL Server Network Configuration* → *Protocols for MSSQLSERVER* (hoặc *SQLEXPRESS*) → bật **TCP/IP** → khởi động lại dịch vụ SQL Server.

- **Bật đăng nhập bằng tài khoản SQL:** nếu lúc cài lỡ chọn *Windows Authentication*, vào SSMS → chuột phải tên server → *Properties* → *Security* → chọn *SQL Server and Windows Authentication mode* → khởi động lại dịch vụ.

Với bản **Express**, cổng thường không phải 1433 cố định. Trong phần cài đặt TCP/IP (tab *IP Addresses*, mục *IPAll*), đặt **TCP Port = 1433** để dễ kết nối.

Cách B — Chạy SQL Server bằng Docker (Windows, macOS, Linux)

Nếu máy đã có Docker Desktop, chỉ cần một lệnh:

```
docker run -e "ACCEPT_EULA=Y" -e "MSSQL_SA_PASSWORD=Matkhau@123" -p 1433:1433
--name sqlserver -d mcr.microsoft.com/mssql/server:2022-latest
```

- `ACCEPT_EULA=Y`: đồng ý điều khoản sử dụng.
- `MSSQL_SA_PASSWORD`: mật khẩu tài khoản `sa`. Mật khẩu phải **đủ mạnh**: tối thiểu 8 ký tự, có ít nhất 3 trong 4 loại (chữ hoa, chữ thường, số, ký tự đặc biệt). Mật khẩu yếu khiến container tự tắt ngay sau khi chạy.
- `-p 1433:1433`: mở cổng 1433 (cổng mặc định của SQL Server).
- Kiểm tra container đang chạy: `docker ps`. Nếu nó tự tắt, xem lý do bằng `docker logs sqlserver`.
- Image SQL Server được build cho chip Intel/AMD. Trên máy Mac chip Apple, cần bật tùy chọn chạy giả lập (Rosetta) trong Docker Desktop.
- Có thể đã có image của phiên bản SQL Server mới hơn; bản 2022 trong lệnh trên hoàn toàn đủ để học.

1.4 Tạo database đầu tiên

Kết nối tới SQL Server bằng SSMS hoặc extension SQL Server trong VS Code với thông tin:

Thông tin	Giá trị
Server	`localhost` (hoặc `localhost,1433`)
Authentication	SQL Login / SQL Server Authentication
User	`sa`
Password	mật khẩu bạn đã đặt
Trust server certificate	Bật (chỉ ở máy local)

Chạy câu lệnh:

```
CREATE DATABASE TaskDb;
```

Thấy database `TaskDb` trong danh sách là thành công.

1.5 Cài Nest CLI và tạo dự án

```
npx @nestjs/cli new task-api --strict
cd task-api
npm run start:dev
```

- Khi được hỏi trình quản lý gói, chọn `npm`.
- Mở trình duyệt vào `http://localhost:3000`: thấy chữ **Hello World!** là đã thành công.
- Dừng ứng dụng bằng `Ctrl + C`.

Nếu `npm install` báo lỗi lạ kiểu `Cannot read properties of null (reading 'edgesOut')`, hãy cập nhật npm (`npm install -g npm@latest`) rồi cài lại.

Tự luyện

- [ ] `node -v` hiện bản 20 trở lên
- [ ] Kết nối được SQL Server bằng tài khoản `sa` và tạo được database `TaskDb`
- [ ] Mở được `http://localhost:3000` thấy *Hello World!*

Bài 2 — TypeScript tối thiểu để học NestJS

TypeScript = JavaScript + **kiểu dữ liệu**. Bạn khai báo biến này là số, chuỗi hay object nào; trình soạn code sẽ báo lỗi **ngay khi gõ** nếu dùng sai, thay vì đợi chạy mới biết. Bài này chỉ dạy đúng những gì cần cho NestJS.

2.1 Khai báo kiểu

```
let age: number = 20;
let name: string = 'An';
let isDone: boolean = false;
let tags: string[] = ['nestjs', 'sql'];

function add(a: number, b: number): number {
  return a + b;
}
add(1, '2'); // ❌ Lỗi ngay khi gõ: '2' là chuỗi, không phải số

let email: string | null = null;      // có thể là chuỗi HOẶC null
// note?: string;                     // dấu ? = không bắt buộc (dùng trong
object/class)
```

2.2 Interface — mô tả hình dạng của object

```
interface Task {
  id: string;
  title: string;
  done: boolean;
}

const t: Task = { id: '1', title: 'Học TypeScript', done: false }; // ✅
const x: Task = { id: '2', title: 'Thiếu done' };                  // ❌
thiếu trường done
```

2.3 Class — thứ NestJS dùng nhiều nhất

```
class Counter {
  private count = 0;          // private: chỉ dùng được bên trong class

  increase(): number {
    this.count++;
    return this.count;
  }
}

const c = new Counter();
c.increase(); // 1
```

Cách viết constructor gọn mà bạn sẽ thấy trong **mọi** file NestJS:

```
class TasksController {
  // Vừa khai báo thuộc tính tasksService, vừa gán giá trị nhận được — chỉ
trong một dòng
  constructor(private readonly tasksService: TasksService) {}
}
// readonly: không gán lại được sau khi khởi tạo
```

2.4 Decorator — những dòng bắt đầu bằng @

```
@Controller('tasks')
export class TasksController {
  @Get()
  findAll() { ... }
}
```

Decorator là một "nhãn dán" gắn lên class hoặc method. Nhãn đó **mô tả** class/method cho NestJS biết. Ở ví dụ trên:

- `@Controller('tasks')` báo: "class này xử lý các đường dẫn bắt đầu bằng `/tasks`".
- `@Get()` báo: "method này chạy khi có request GET".

Khi khởi động, NestJS đọc các nhãn này để tự nối mọi thứ lại với nhau. Bạn **chưa cần biết cách tự viết decorator**, chỉ cần biết dùng.

2.5 async/await với kiểu Promise

```
async function getTask(id: string): Promise<Task> {
  const task = await database.findTask(id); // chờ database trả kết quả
  return task;
}
```

`Promise<Task>` nghĩa là: "hàm này trả về Task, nhưng **trong tương lai**". Mọi thao tác với database đều là bất đồng bộ, nên bạn sẽ thấy `async/await` liên tục.

2.6 Import và export

```
// tasks.service.ts
export class TasksService { ... }

// tasks.controller.ts
import { TasksService } from './tasks.service.js'; // ⚠️ NestJS 12: ghi đuôi
.js dù file là .ts
```

Lưu ý: dù file thật là `.ts`, trong NestJS 12 bạn vẫn ghi `.js` khi import file của chính dự án. Lý do là code sẽ được biên dịch ra file `.js` rồi mới chạy. Quên đuôi `.js` sẽ gặp lỗi *Cannot find module*.

Tự luyện

1. Viết `interface User` có `id`, `email`, `createdAt` (kiểu `Date`).
2. Viết class `Calculator` có method `add`, `subtract` với tham số kiểu `number`.
3. Cố tình truyền chuỗi vào `add` để thấy VS Code gạch đỏ báo lỗi.

Bài 3 — Dự án đầu tiên: cấu trúc thư mục và controller

3.1 Dự án vừa tạo có gì?

```
task-api/
├── src/
│   ├── main.ts                ← điểm khởi động ứng dụng
│   ├── app.module.ts          ← module gốc, "gom" mọi phần của ứng dụng
│   ├── app.controller.ts      ← controller mẫu (trả "Hello World!")
│   ├── app.service.ts         ← service mẫu
│   └── app.controller.spec.ts← file test mẫu
├── test/                      ← test toàn bộ ứng dụng (e2e)
├── package.json               ← danh sách thư viện và lệnh chạy
├── tsconfig.json              ← cấu hình TypeScript
└── nest-cli.json              ← cấu hình Nest CLI
```

```
// src/main.ts
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module.js';

async function bootstrap() {
  const app = await NestFactory.create(AppModule); // tạo ứng dụng từ module
gốc
  await app.listen(process.env.PORT ?? 3000);      // chạy ở cổng 3000
}
await bootstrap();
```

Các lệnh hay dùng:

Lệnh	Tác dụng
`npm run start:dev`	Chạy và **tự khởi động lại** mỗi khi bạn lưu file (dùng khi học)
`npm run build`	Biên dịch TypeScript ra thư mục `dist/`
`npm run start:prod`	Chạy bản đã biên dịch
`npm test`	Chạy unit test

3.2 Controller là gì?

Controller nhận request từ client và trả response. Mỗi method trong controller xử lý một đường dẫn (route) cụ thể.

Tạo module và controller cho tính năng "công việc" bằng CLI:

```
nest g module tasks       # tạo src/tasks/tasks.module.ts và tự khai báo vào
AppModule
nest g controller tasks   # tạo src/tasks/tasks.controller.ts và tự khai báo
vào TasksModule
```

Nếu máy chưa cài lệnh `nest` toàn cục, dùng `npx nest g module tasks`.

3.3 Viết các route đầu tiên

Tạm thời lưu dữ liệu trong một mảng (chưa dùng database):

```
// src/tasks/tasks.controller.ts
import { Body, Controller, Delete, Get, HttpCode, NotFoundException, Param,
Post } from '@nestjs/common';

@Controller('tasks') // mọi route trong class này bắt đầu bằng /tasks
export class TasksController {
  private tasks = [
    { id: 1, title: 'Cài Node.js', done: true },
    { id: 2, title: 'Học NestJS', done: false },
  ];

  @Get() // GET /tasks
  findAll() {
    return this.tasks; // NestJS tự chuyển thành JSON
  }

  @Get(':id') // GET /tasks/1
  findOne(@Param('id') id: string) {
    const task = this.tasks.find((t) => t.id === Number(id));
    if (!task) throw new NotFoundException('Không tìm thấy công việc'); //
trả lỗi 404
    return task;
  }

  @Post() // POST /tasks — body: { "title": "..." }
  create(@Body() body: { title: string }) {
    const task = { id: Date.now(), title: body.title, done: false };
    this.tasks.push(task);
    return task; // POST mặc định trả mã 201 Created
  }

  @Delete(':id') // DELETE /tasks/1
  @HttpCode(204) // 204: thành công, không trả nội dung
  remove(@Param('id') id: string) {
    this.tasks = this.tasks.filter((t) => t.id !== Number(id));
  }
}
```

Decorator	Ý nghĩa
`@Controller('tasks')`	Tiền tố đường dẫn `/tasks`

Decorator	Ý nghĩa
`@Get()`, `@Post()`, `@Patch()`, `@Put()`, `@Delete()`	Phương thức HTTP
`@Param('id')`	Lấy giá trị `:id` trên URL. Luôn là **chuỗi**
`@Body()`	Lấy dữ liệu JSON gửi lên
`@Query('keyword')`	Lấy `?keyword=...` trên URL
`@HttpCode(204)`	Đổi mã trạng thái trả về

3.4 Thử API

Có thể dùng **Postman**, extension **Thunder Client** trong VS Code, hoặc lệnh `curl`:

```
curl http://localhost:3000/tasks
curl http://localhost:3000/tasks/1
curl -X POST http://localhost:3000/tasks -H "Content-Type: application/json"
-d "{\"title\":\"Viết API đầu tiên\"}"
curl -X DELETE http://localhost:3000/tasks/1
```

Đến Bài 10 bạn sẽ có Swagger: thử API ngay trên trình duyệt, tiện hơn nhiều.

Để ý: mảng `tasks` nằm trong bộ nhớ, nên khởi động lại ứng dụng là dữ liệu mất hết. Bài 7 sẽ lưu vào SQL Server.

Tự luyện

1. Thêm route `GET /tasks/count` trả về số công việc. Đặt method này **phía trên** `@Get(':id')`, vì nếu đặt dưới thì NestJS sẽ hiểu `count` là một `id`.
2. Thêm `GET /tasks?done=true` chỉ trả về công việc đã xong (dùng `@Query('done')`).
3. Gọi `GET /tasks/999` và xem lỗi 404 trông như thế nào.

Bài 4 — Service, module và dependency injection

4.1 Vì sao cần service?

Ở Bài 3, controller vừa nhận request vừa xử lý dữ liệu. Dự án lớn lên thì controller sẽ rất rối. NestJS chia việc rõ ràng:

Thành phần	Việc của nó	Ví dụ đời thường
Controller	Nhận request, gọi service, trả response	Nhân viên lễ tân nhận yêu cầu
Service	Xử lý nghiệp vụ, làm việc với dữ liệu	Đầu bếp nấu món
Module	Gom controller + service của một tính năng	Một gian bếp riêng

4.2 Tạo service

```
nest g service tasks   # tạo src/tasks/tasks.service.ts và tự khai báo vào TasksModule
```

```
// src/tasks/tasks.service.ts
import { Injectable, NotFoundException } from '@nestjs/common';

@Injectable() // đánh dấu: NestJS được phép tạo và "tiêm" class này vào nơi khác
export class TasksService {
  private tasks = [{ id: 1, title: 'Học NestJS', done: false }];

  findAll() {
    return this.tasks;
  }

  findOne(id: number) {
    const task = this.tasks.find((t) => t.id === id);
    if (!task) throw new NotFoundException('Không tìm thấy công việc');
    return task;
  }

  create(title: string) {
    const task = { id: Date.now(), title, done: false };
    this.tasks.push(task);
    return task;
  }
}
```

```
// src/tasks/tasks.controller.ts — controller giờ rất gọn
import { Body, Controller, Get, Param, ParseIntPipe, Post } from
'@nestjs/common';
import { TasksService } from './tasks.service.js';

@Controller('tasks')
export class TasksController {
  constructor(private readonly tasksService: TasksService) {} // ← dependency
injection

  @Get()
  findAll() {
    return this.tasksService.findAll();
  }

  @Get(':id')
  findOne(@Param('id', ParseIntPipe) id: number) { // ParseIntPipe: đổi "1"
thành số 1, sai thì báo 400
    return this.tasksService.findOne(id);
  }

  @Post()
  create(@Body() body: { title: string }) {
    return this.tasksService.create(body.title);
  }
}
```

4.3 Dependency injection (DI) — hiểu đơn giản

Để ý dòng `constructor(private readonly tasksService: TasksService) {}`. Bạn **không hề viết** `new TasksService()`. Vậy service từ đâu ra? Quy trình như sau:

1. Lúc khởi động, NestJS đọc constructor và thấy controller **cần** một `TasksService`.
2. NestJS tự tạo **một** `TasksService` và đưa ("tiêm") vào controller.
3. Nơi nào khác cũng cần `TasksService` thì NestJS đưa **cùng một bản đó**.

Lợi ích:

- Code không phải tự lo việc tạo đối tượng.
- Khi viết test, ta dễ dàng đưa vào một service "giả" (Bài 10).

4.4 Module

```
// src/tasks/tasks.module.ts
import { Module } from '@nestjs/common';
import { TasksController } from './tasks.controller.js';
import { TasksService } from './tasks.service.js';

@Module({
  controllers: [TasksController], // controller của module này
  providers: [TasksService],      // service của module này (NestJS sẽ tạo và
tiêm)
  exports: [TasksService],        // (tùy chọn) cho module KHÁC được dùng
TasksService
})
export class TasksModule {}

// src/app.module.ts — module gốc import các module tính năng
@Module({
  imports: [TasksModule],
})
export class AppModule {}
```

Quy tắc cần nhớ: service mặc định chỉ dùng được **trong module của nó**. Muốn `AuthModule` dùng `UsersService` thì phải làm đủ hai việc:

1. `UsersModule` ghi `exports: [UsersService]`.
2. `AuthModule` ghi `imports: [UsersModule]`.

4.5 Lỗi bạn chắc chắn sẽ gặp

```
Nest can't resolve dependencies of the TasksController (?).
Please make sure that the argument TasksService at index [0] is available in
the TasksModule context.
```

Nghĩa là NestJS không tìm thấy `TasksService` để tiêm. Kiểm tra theo thứ tự:

- `TasksService` đã có trong `providers` của module chưa?
- Nếu service thuộc module khác: module đó đã `exports` chưa, và module này đã `imports` chưa?
- Class service có `@Injectable()` không?

Tự luyện

1. Thêm method `remove(id)` vào service và route `DELETE /tasks/:id` gọi nó.
2. Xóa `TasksService` khỏi `providers`, chạy lại để thấy lỗi ở mục 4.5, rồi sửa lại.
3. Gọi `GET /tasks/abc` và giải thích vì sao nhận lỗi 400.

Bài 5 — DTO và kiểm tra dữ liệu đầu vào

Nguyên tắc vàng: không bao giờ tin dữ liệu người dùng gửi lên. Họ có thể gửi thiếu trường, sai kiểu, chuỗi dài 1 triệu ký tự, hoặc cố tình gửi thêm trường lạ. Bài này dạy cách chặn tất cả những trường hợp đó, chỉ với vài dòng.

5.1 DTO là gì?

DTO (Data Transfer Object) là một class mô tả **dữ liệu được phép gửi lên** cho một API, kèm quy tắc kiểm tra cho từng trường.

```
npm install class-validator class-transformer
```

```
// src/tasks/dto/create-task.dto.ts
import { IsNotEmpty, IsString, MaxLength } from 'class-validator';

export class CreateTaskDto {
  @IsString()
  @IsNotEmpty({ message: 'Tiêu đề không được để trống' })
  @MaxLength(200, { message: 'Tiêu đề tối đa 200 ký tự' })
  title: string;
}
```

```
// src/tasks/dto/update-task.dto.ts
import { PartialType } from '@nestjs/mapped-types'; // Bài 10 sẽ đổi sang
'@nestjs/swagger'
import { IsBoolean, IsOptional } from 'class-validator';
import { CreateTaskDto } from './create-task.dto.js';

// PartialType: lấy mọi trường của CreateTaskDto nhưng biến thành KHÔNG bắt
buộc
export class UpdateTaskDto extends PartialType(CreateTaskDto) {
  @IsOptional() // có thể không gửi
  @IsBoolean({ message: 'done phải là true hoặc false' })
  done?: boolean;
}
```

Cài thêm `@nestjs/mapped-types` nếu dự án chưa có: `npm install @nestjs/mapped-types`.

DTO phải là `class`, không được là `interface`. Lý do: interface bị xóa mất khi biên dịch, nên NestJS không còn gì để đọc quy tắc kiểm tra.

5.2 Bật kiểm tra cho toàn ứng dụng

```
// src/main.ts
import { ValidationPipe } from '@nestjs/common';

const app = await NestFactory.create(AppModule);
app.useGlobalPipes(
  new ValidationPipe({
    whitelist: true,            // tự bỏ các trường không khai báo trong DTO
    forbidNonWhitelisted: true,  // ...và báo lỗi nếu có trường lạ
    transform: true,            // chuyển dữ liệu thành đúng class DTO, đúng
kiểu
  }),
);
```

5.3 Dùng DTO trong controller

```
@Post()
create(@Body() dto: CreateTaskDto) {        // thay vì body: { title: string
}
  return this.tasksService.create(dto.title);
}

@Patch(':id')
update(@Param('id', ParseIntPipe) id: number, @Body() dto: UpdateTaskDto) {
  return this.tasksService.update(id, dto);
}
```

Thử gửi dữ liệu sai: `POST /tasks` với body `{"title": "", "role": "admin"}`. Kết quả trả về dạng như sau (thứ tự các thông báo có thể khác):

```
{
  "message": ["property role should not exist", "Tiêu đề không được để trống"],
  "error": "Bad Request",
  "statusCode": 400
}
```

Code trong service **không phải viết thêm dòng kiểm tra nào**. Request sai bị chặn từ trước khi tới controller.

5.4 Các quy tắc kiểm tra hay dùng

Decorator	Kiểm tra
`@IsString()`, `@IsInt()`, `@IsNumber()`, `@IsBoolean()`	Đúng kiểu
`@IsNotEmpty()`	Không rỗng
`@IsOptional()`	Không bắt buộc (nếu có thì vẫn kiểm tra các quy tắc khác)
`@MinLength(n)`, `@MaxLength(n)`	Độ dài chuỗi
`@Min(n)`, `@Max(n)`	Giá trị số
`@IsEmail()`	Đúng định dạng email

Decorator	Kiểm tra
`@IsUUID()`	Đúng định dạng UUID
`@IsIn(['low', 'medium', 'high'])`	Chỉ nhận các giá trị trong danh sách
`@IsDateString()`	Chuỗi ngày giờ hợp lệ, ví dụ `2026-12-31`

Mỗi decorator đều nhận `{ message: '...' }` để viết thông báo lỗi tiếng Việt.

Tự luyện

1. Thêm trường `priority` vào `CreateTaskDto`: không bắt buộc, chỉ nhận `low`, `medium`, `high`.
2. Viết `update(id, dto)` trong service: tìm task, gộp dữ liệu mới bằng `Object.assign(task, dto)`.
3. Gửi `{"done": "yes"}` vào `PATCH /tasks/1` và đọc thông báo lỗi.

Bài 6 — SQL Server cơ bản

Trước khi để NestJS làm việc với SQL Server, bạn cần hiểu dữ liệu được lưu thế nào. Ngôn ngữ truy vấn của SQL Server gọi là **T-SQL** (Transact-SQL).

Bài này thực hành trên một database riêng tên `PracticeDb`, để database `TaskDb` dành cho NestJS ở Bài 7. Chạy các câu lệnh bằng SSMS hoặc extension SQL Server trong VS Code: bôi đen câu lệnh, bấm **Execute** (SSMS: phím `F5`).

6.1 Khái niệm cơ bản

Khái niệm	Giải thích	Ví dụ
Database	Một kho dữ liệu	`PracticeDb`
Bảng (table)	Giống một trang Excel, chứa một loại dữ liệu	`Users`, `Tasks`
Cột (column)	Một thuộc tính, có kiểu dữ liệu cố định	`Email NVARCHAR(255)`
Hàng (row)	Một bản ghi	Một người dùng cụ thể

Khái niệm	Giải thích	Ví dụ
Khóa chính (primary key)	Cột định danh duy nhất mỗi hàng	`Id`
Khóa ngoại (foreign key)	Cột trỏ sang khóa chính của bảng khác	`Tasks.UserId` → `Users.Id`

6.2 Kiểu dữ liệu hay dùng

Kiểu	Dùng cho	Ghi chú
`INT`, `BIGINT`	Số nguyên	
`DECIMAL(18,2)`	Tiền, số thập phân chính xác	Không dùng `FLOAT` cho tiền
`NVARCHAR(n)`	**Chuỗi có tiếng Việt**	Chữ **N** = Unicode. `VARCHAR` thường có thể làm hỏng chữ có dấu
`NVARCHAR(MAX)`	Văn bản dài	
`BIT`	Đúng/sai (0 hoặc 1)	SQL Server **không có** kiểu `BOOLEAN`
`DATETIME2`	Ngày giờ	Nên dùng thay cho `DATETIME` cũ
`UNIQUEIDENTIFIER`	Mã định danh dạng GUID/UUID	Ví dụ `6F9619FF-8B86-D011-B42D-00C04FC964FF`

6.3 Tạo database và bảng

```
CREATE DATABASE PracticeDb;
GO
USE PracticeDb;
GO

CREATE TABLE Users (
  Id        UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID() PRIMARY KEY, -- NEWID()
tự sinh mã
  Email     NVARCHAR(255)    NOT NULL UNIQUE,                      -- không
cho trùng email
  FullName  NVARCHAR(100)    NULL,
  CreatedAt DATETIME2        NOT NULL DEFAULT SYSUTCDATETIME()     -- giờ
```

```
hiện tại (UTC)
);

CREATE TABLE Tasks (
  Id        INT IDENTITY(1,1) PRIMARY KEY,        -- IDENTITY: số tự tăng 1,
2, 3...
  Title     NVARCHAR(200)    NOT NULL,
  Done      BIT              NOT NULL DEFAULT 0,
  UserId    UNIQUEIDENTIFIER NOT NULL,
  CreatedAt DATETIME2        NOT NULL DEFAULT SYSUTCDATETIME(),
  CONSTRAINT FK_Tasks_Users FOREIGN KEY (UserId)
    REFERENCES Users(Id) ON DELETE CASCADE      -- xóa user thì xóa luôn task
của user đó
);
```

- `GO` **không phải** câu lệnh T-SQL. Nó là dấu ngắt mà SSMS dùng để chia các lệnh thành từng đợt chạy; trong code ứng dụng không dùng `GO`.
- Bảng `Users` dùng `UNIQUEIDENTIFIER` (mã ngẫu nhiên, khó đoán), bảng `Tasks` dùng `INT IDENTITY` (số tự tăng). Đây chỉ là để bạn thấy cả hai cách; từ Bài 7, cả hai bảng đều dùng mã UUID.

6.4 Thêm, xem, sửa, xóa (CRUD)

```
-- THÊM: chuỗi tiếng Việt phải có tiền tố N'...'
INSERT INTO Users (Email, FullName) VALUES (N'an@example.com', N'Nguyễn Văn
An');

DECLARE @UserId UNIQUEIDENTIFIER = (SELECT Id FROM Users WHERE Email =
N'an@example.com');
INSERT INTO Tasks (Title, UserId) VALUES
  (N'Học SQL Server', @UserId),
  (N'Học NestJS', @UserId);

-- XEM
SELECT * FROM Tasks;                                    -- tất cả cột, tất cả
hàng
SELECT Id, Title FROM Tasks WHERE Done = 0;             -- lọc việc chưa xong
SELECT TOP 5 * FROM Tasks ORDER BY CreatedAt DESC;      -- 5 việc mới nhất
SELECT * FROM Tasks WHERE Title LIKE N'%NestJS%';       -- tìm theo từ khóa

-- Phân trang: bỏ qua 10 hàng đầu, lấy 10 hàng tiếp (trang 2)
SELECT * FROM Tasks ORDER BY Id OFFSET 10 ROWS FETCH NEXT 10 ROWS ONLY;
```

```
-- SỬA (luôn có WHERE!)
UPDATE Tasks SET Done = 1 WHERE Id = 1;

-- XÓA (luôn có WHERE!)
DELETE FROM Tasks WHERE Id = 2;
```

Hai lưu ý rất quan trọng:

- **Quên tiền tố `N`** trước chuỗi tiếng Việt (`'Nguyễn'` thay vì `N'Nguyễn'`) có thể khiến chữ có dấu bị lưu thành dấu `?`.
- **Quên `WHERE` trong `UPDATE` / `DELETE`** sẽ sửa hoặc xóa **toàn bộ bảng**. Thói quen tốt: viết `SELECT` với điều kiện `WHERE` trước để xem đúng hàng chưa, rồi mới đổi thành `UPDATE` / `DELETE`.

6.5 Kết hợp nhiều bảng (JOIN) và thống kê

```
-- Danh sách công việc kèm email người tạo
SELECT t.Title, t.Done, u.Email
FROM Tasks t
JOIN Users u ON u.Id = t.UserId;

-- Mỗi người có bao nhiêu việc, bao nhiêu việc đã xong
SELECT u.Email,
       COUNT(t.Id) AS TongSo,
       SUM(CASE WHEN t.Done = 1 THEN 1 ELSE 0 END) AS DaXong
FROM Users u
LEFT JOIN Tasks t ON t.UserId = u.Id    -- LEFT JOIN: giữ cả người chưa có
việc nào
GROUP BY u.Email;
```

6.6 Thử ràng buộc của database

```
INSERT INTO Users (Email) VALUES (N'an@example.com');
-- Lỗi 2627: Violation of UNIQUE KEY constraint ... Cannot insert duplicate
key

INSERT INTO Tasks (Title, UserId) VALUES (N'Việc mồ côi', NEWID());
-- Lỗi 547: The INSERT statement conflicted with the FOREIGN KEY constraint
...
```

Ràng buộc giúp database **tự bảo vệ** dữ liệu, kể cả khi code có lỗi. Mã lỗi **2627** (trùng dữ liệu) sẽ được dùng lại ở Bài 8 để trả thông báo dễ hiểu cho người dùng.

Tự luyện

1. Thêm cột `DueDate DATETIME2 NULL` vào bảng `Tasks` bằng `ALTER TABLE Tasks ADD DueDate DATETIME2 NULL;`.
2. Viết câu lệnh lấy các việc chưa xong của `an@example.com`, mới nhất trước.
3. Xóa user `an@example.com` rồi kiểm tra: các task của user đó còn không? Vì sao?

Bài 7 — Kết nối NestJS với SQL Server bằng TypeORM

TypeORM là thư viện giúp bạn làm việc với database bằng class TypeScript thay vì tự viết từng câu SQL.

Khái niệm	Là gì
Entity	Class mô tả một bảng
Repository	Đối tượng có sẵn các hàm `find`, `save`, `remove`... để thao tác với bảng đó

7.1 Cài thư viện

```
npm install @nestjs/typeorm typeorm mssql @nestjs/config
```

- `mssql`: "driver" giúp Node.js nói chuyện được với SQL Server.
- `@nestjs/config`: đọc cấu hình từ file `.env`.

7.2 File `.env` — nơi để thông tin kết nối

```
# .env (đặt ở thư mục gốc dự án)
PORT=3000
DB_HOST=localhost
DB_PORT=1433
DB_USERNAME=sa
DB_PASSWORD=Matkhau@123
DB_NAME=TaskDb
JWT_SECRET=thay-bang-chuoi-bi-mat-dai-it-nhat-32-ky-tu
```

Thêm `.env` vào file `.gitignore` để **không bao giờ đưa mật khẩu lên GitHub**. Tạo thêm file `.env.example` có cùng tên biến nhưng giá trị giả, để người khác biết cần cấu hình gì.

7.3 Cấu hình kết nối trong `AppModule`

```
// src/app.module.ts
import { Module } from '@nestjs/common';
import { ConfigModule, ConfigService } from '@nestjs/config';
import { TypeOrmModule } from '@nestjs/typeorm';
import { TasksModule } from './tasks/tasks.module.js';

@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true }), // đọc file .env, dùng được ở
mọi nơi
    TypeOrmModule.forRootAsync({
      inject: [ConfigService],
      useFactory: (config: ConfigService) => ({
        type: 'mssql',
        host: config.getOrThrow<string>('DB_HOST'),
        port: Number(config.get('DB_PORT') ?? 1433),
        username: config.getOrThrow<string>('DB_USERNAME'),
        password: config.getOrThrow<string>('DB_PASSWORD'),
        database: config.getOrThrow<string>('DB_NAME'),
        options: {
          encrypt: true,
          trustServerCertificate: true, // CHỈ dùng ở máy local (chứng chỉ
SQL Server tự ký)
        },
        autoLoadEntities: true, // tự nạp các entity đã khai báo trong
forFeature()
        synchronize: true,      // tự tạo/sửa bảng theo entity — CHỈ dùng khi
học ở máy local
      }),
    }),
    TasksModule,
  ],
})
export class AppModule {}
```

- `getOrThrow` báo lỗi ngay khi khởi động nếu thiếu biến trong `.env`, giúp bạn biết sớm thay vì lỗi khó hiểu về sau.

- Dùng `forRootAsync` (thay vì `forRoot`) vì cấu hình cần đọc từ `ConfigService`.

⚠️ **Về `synchronize: true`:** TypeORM sẽ tự tạo và sửa bảng cho khớp với entity mỗi lần khởi động, rất tiện khi học. Nhưng nó có thể **xóa cột và làm mất dữ liệu** khi bạn đổi entity. **Không bao giờ bật trên database thật của công ty.** Khi đi làm, mọi thay đổi bảng được thực hiện bằng *migration* — file mô tả thay đổi, được kiểm tra và chạy có kiểm soát. Bạn sẽ học migration ở giáo trình nâng cao.

7.4 Entity — class mô tả bảng

```
// src/tasks/task.entity.ts
import { Column, CreateDateColumn, Entity, PrimaryGeneratedColumn } from
'typeorm';

@Entity('Tasks') // tên bảng trong SQL Server
export class Task {
  @PrimaryGeneratedColumn('uuid') // khóa chính kiểu UNIQUEIDENTIFIER, tự
sinh
  id: string;

  @Column({ type: 'nvarchar', length: 200 }) // NVARCHAR để lưu tiếng Việt
  title: string;

  @Column({ type: 'bit', default: false }) // SQL Server dùng BIT cho
đúng/sai
  done: boolean;

  @CreateDateColumn({ type: 'datetime2' }) // tự điền thời điểm tạo
  createdAt: Date;
}
```

TypeScript	Kiểu trong SQL Server
`string`	`nvarchar` (mặc định, độ dài 255 nếu không ghi `length`)
`boolean`	`bit`
`number`	`int` (ghi rõ `type: 'decimal'` cho tiền)
`Date`	`datetime2` khi ghi rõ `type: 'datetime2'`
khóa `'uuid'`	`uniqueidentifier`

7.5 Đăng ký entity và dùng Repository

```
// src/tasks/tasks.module.ts
@Module({
  imports: [TypeOrmModule.forFeature([Task])], // cho module này dùng
Repository<Task>
  controllers: [TasksController],
  providers: [TasksService],
})
export class TasksModule {}
```

```
// src/tasks/tasks.service.ts
import { Injectable, NotFoundException } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { CreateTaskDto } from './dto/create-task.dto.js';
import { UpdateTaskDto } from './dto/update-task.dto.js';
import { Task } from './task.entity.js';

@Injectable()
export class TasksService {
  constructor(@InjectRepository(Task) private readonly tasksRepo:
Repository<Task>) {}

  findAll() {
    return this.tasksRepo.find({ order: { createdAt: 'DESC' } });  // SELECT
... ORDER BY createdAt DESC
  }
```

```
  async findOne(id: string) {
    const task = await this.tasksRepo.findOneBy({ id });           // SELECT
... WHERE id = @0
    if (!task) throw new NotFoundException('Không tìm thấy công việc');
    return task;
  }

  create(dto: CreateTaskDto) {
    const task = this.tasksRepo.create(dto); // chỉ tạo object, CHƯA lưu
    return this.tasksRepo.save(task);        // INSERT INTO Tasks ...
  }

  async update(id: string, dto: UpdateTaskDto) {
    const task = await this.findOne(id);
    Object.assign(task, dto);                // gộp dữ liệu mới vào
    return this.tasksRepo.save(task);        // đã có id → UPDATE
  }

  async remove(id: string) {
    const task = await this.findOne(id);
    await this.tasksRepo.remove(task);       // DELETE
  }
}
```

Trong controller, đổi `ParseIntPipe` thành **`ParseUUIDPipe`** vì id giờ là UUID:

```
@Get(':id')
findOne(@Param('id', ParseUUIDPipe) id: string) {
  return this.tasksService.findOne(id);
}
```

TypeORM luôn truyền giá trị dưới dạng **tham số** (`@0`, `@1` …) chứ không ghép thẳng vào câu SQL, nên bạn được bảo vệ khỏi lỗi **SQL Injection**.

7.6 Chạy và kiểm tra

1. `npm run start:dev`.
2. Mở SSMS, refresh database `TaskDb`: thấy bảng `Tasks` do TypeORM tự tạo.
3. Gọi `POST /tasks` với `{"title": "Học SQL Server với NestJS"}`, rồi chạy `SELECT * FROM Tasks` trong SSMS để thấy dữ liệu.

Một điểm lạ có thể gặp: SQL Server thường trả mã `UNIQUEIDENTIFIER` ở dạng **chữ in hoa**, như `6F9619FF-...`. Điều này bình thường; SQL Server so sánh mã này không phân biệt hoa thường.

7.7 Lỗi kết nối thường gặp

Thông báo	Nguyên nhân	Cách sửa
`Unable to connect to the database. Retrying...` kèm `Failed to connect to localhost:1433 - Could not connect`	SQL Server chưa chạy, chưa bật TCP/IP, hoặc sai cổng	Kiểm tra dịch vụ/container đang chạy; bật TCP/IP (Bài 1.3); kiểm tra `DB_PORT`
`Login failed for user 'sa'`	Sai mật khẩu, hoặc server chưa bật đăng nhập bằng tài khoản SQL	Kiểm tra `DB_PASSWORD`; bật Mixed Mode (Bài 1.3)
Lỗi chứng chỉ (`self-signed certificate`...)	SQL Server local dùng chứng chỉ tự ký	`trustServerCertificate: true` (chỉ ở máy local)
`Cannot open database "TaskDb"`	Chưa tạo database	Chạy `CREATE DATABASE TaskDb;`

Tự luyện

1. Thêm cột `description` (`nvarchar`, độ dài 1000, cho phép trống: `nullable: true`). Xem TypeORM tự thêm cột trong SSMS.
2. Thêm route `GET /tasks?done=true` dùng `this.tasksRepo.find({ where: { done: true } })`.
3. Tắt SQL Server rồi chạy lại ứng dụng để nhận diện lỗi kết nối.

Bài 8 — Quan hệ giữa các bảng và xử lý lỗi

Trong thực tế, mỗi công việc thuộc về một người dùng. Đây là quan hệ **một – nhiều**: một `User` có nhiều `Task`, mỗi `Task` thuộc đúng một `User`. Bạn đã làm quan hệ này bằng SQL ở Bài 6; giờ làm bằng TypeORM.

8.1 Entity `User`

```
nest g module users
nest g service users
```

```
// src/users/user.entity.ts
import { Exclude } from 'class-transformer';
import { Column, CreateDateColumn, Entity, OneToMany, PrimaryGeneratedColumn
} from 'typeorm';
import type { Relation } from 'typeorm';
import { Task } from '../tasks/task.entity.js';

@Entity('Users')
export class User {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column({ type: 'nvarchar', length: 255, unique: true }) // không cho trùng
email
  email: string;

  @Exclude() // không bao giờ trả ra ngoài API (xem 8.4)
  @Column({ type: 'nvarchar', length: 255 })
  passwordHash: string;

  @OneToMany(() => Task, (task) => task.user) // một user có NHIỀU task
  tasks: Relation<Task[]>;

  @CreateDateColumn({ type: 'datetime2' })
  createdAt: Date;
}
```

8.2 Thêm quan hệ vào `Task`

```
// src/tasks/task.entity.ts — phiên bản đầy đủ
import { Column, CreateDateColumn, Entity, JoinColumn, ManyToOne,
PrimaryGeneratedColumn } from 'typeorm';
import type { Relation } from 'typeorm';
import { User } from '../users/user.entity.js';

@Entity('Tasks')
export class Task {
```

```
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column({ type: 'nvarchar', length: 200 })
  title: string;

  @Column({ type: 'bit', default: false })
  done: boolean;

  @Column({ type: 'uniqueidentifier' }) // cột khóa ngoại, lưu id người dùng
  userId: string;

  @ManyToOne(() => User, (user) => user.tasks, { onDelete: 'CASCADE' }) //
NHIỀU task thuộc MỘT user
  @JoinColumn({ name: 'userId' })                                        //
dùng cột userId ở trên
  user: Relation<User>;

  @CreateDateColumn({ type: 'datetime2' })
  createdAt: Date;
}
```

- `@ManyToOne` đặt ở bảng **chứa khóa ngoại** (`Tasks`). `@OneToMany` ở bảng còn lại là chiều ngược, giúp đọc dữ liệu tiện hơn; nó **không** tạo cột nào.
- `onDelete: 'CASCADE'` : xóa user thì xóa luôn task của họ, giống `ON DELETE CASCADE` ở Bài 6.
- **Vì sao có `Relation<...>` ?** Hai file `user.entity.ts` và `task.entity.ts` import lẫn nhau. Trong dự án NestJS 12, cách viết thông thường có thể gây lỗi khi khởi động. Bọc kiểu bằng `Relation<>` giúp tránh lỗi đó. Bạn chỉ cần nhớ quy tắc: **thuộc tính quan hệ luôn bọc `Relation<>`** , và import nó bằng `import type` .

Sau khi khởi động lại (`synchronize: true`), mở SSMS sẽ thấy bảng `Users` , và bảng `Tasks` có thêm cột `userId` cùng khóa ngoại trỏ sang `Users` .

Lưu ý: nếu bảng `Tasks` **đã có dữ liệu** từ Bài 7, TypeORM không thể thêm cột `userId` bắt buộc (`NOT NULL`) vì các hàng cũ không có giá trị. Ở máy học, cách đơn giản nhất là xóa dữ liệu cũ (`DELETE FROM Tasks;`) rồi khởi động lại.

8.3 Đọc dữ liệu kèm quan hệ

```
// Lấy task kèm thông tin người tạo (TypeORM tự JOIN)
this.tasksRepo.find({ relations: { user: true } });

// Lấy task của một người dùng — chỉ cần lọc theo cột userId, không cần JOIN
this.tasksRepo.find({ where: { userId }, order: { createdAt: 'DESC' } });
```

8.4 Ẩn mật khẩu khỏi kết quả API

Để `@Exclude()` có tác dụng, bật `ClassSerializerInterceptor` cho toàn ứng dụng:

```
// src/app.module.ts
import { ClassSerializerInterceptor } from '@nestjs/common';
import { APP_INTERCEPTOR } from '@nestjs/core';

@Module({
  // imports: [...]
  providers: [{ provide: APP_INTERCEPTOR, useClass:
ClassSerializerInterceptor }],
})
export class AppModule {}
```

Cơ chế này chỉ hoạt động khi bạn trả về **đối tượng entity** lấy từ repository. Nếu bạn tự tạo object mới (`{ ...user }`), `@Exclude()` sẽ bị bỏ qua.

8.5 Xử lý lỗi

Lỗi có sẵn của NestJS. Ném ra ở bất kỳ đâu, NestJS tự trả về mã HTTP tương ứng:

Exception	Mã	Khi nào dùng
`BadRequestException`	400	Dữ liệu gửi lên sai
`UnauthorizedException`	401	Chưa đăng nhập hoặc token sai
`ForbiddenException`	403	Đã đăng nhập nhưng không có quyền
`NotFoundException`	404	Không tìm thấy
`ConflictException`	409	Trùng dữ liệu

Biến lỗi của SQL Server thành thông báo dễ hiểu. Khi đăng ký trùng email, SQL Server báo lỗi số 2627 (đã gặp ở Bài 6.6). Nếu không xử lý, người dùng nhận lỗi 500 khó hiểu. Viết một **exception filter** — lớp bắt lỗi:

```
// src/common/sql-exception.filter.ts
import { ArgumentsHost, Catch, ExceptionFilter, Logger } from
'@nestjs/common';
import type { Response } from 'express';
import { QueryFailedError } from 'typeorm';

// Mã lỗi của SQL Server khi dữ liệu bị trùng (vi phạm UNIQUE / PRIMARY KEY)
const DUPLICATE_ERRORS = [2627, 2601];

@Catch(QueryFailedError) // chỉ bắt lỗi truy vấn database
export class SqlExceptionFilter implements ExceptionFilter {
  private readonly logger = new Logger(SqlExceptionFilter.name);

  catch(exception: QueryFailedError & { driverError?: { number?: number } },
host: ArgumentsHost) {
    const res = host.switchToHttp().getResponse<Response>();
    const errorNumber = exception.driverError?.number;

    if (errorNumber !== undefined && DUPLICATE_ERRORS.includes(errorNumber))
{
      res.status(409).json({ statusCode: 409, message: 'Dữ liệu đã tồn tại'
});
      return;
    }
    this.logger.error(exception.message); // ghi log để lập trình viên xem
    res.status(500).json({ statusCode: 500, message: 'Lỗi cơ sở dữ liệu' });
// không lộ chi tiết
  }
}
```

```
// src/app.module.ts — đăng ký cho toàn ứng dụng
import { APP_FILTER } from '@nestjs/core';

providers: [
  { provide: APP_INTERCEPTOR, useClass: ClassSerializerInterceptor },
  { provide: APP_FILTER, useClass: SqlExceptionFilter },
],
```

- Mã **2627** là trùng ràng buộc `UNIQUE`/khóa chính; **2601** là trùng *unique index*. Thư viện `mssql` đưa mã lỗi vào thuộc tính `number`.
- Nếu gặp lỗi khác chưa xử lý, hãy `console.log(exception.driverError)` để xem số lỗi, rồi bổ sung vào filter.
- **Không bao giờ** trả nguyên thông báo lỗi SQL cho người dùng, vì nó có thể lộ tên bảng, tên cột.

Tự luyện

1. Viết `UsersService` có `create(email, passwordHash)` và `findByEmail(email)`. Đây là đáp án cho Bài 9.
2. Thử thêm hai user trùng email bằng SSMS để thấy lỗi 2627.
3. Thêm vào filter mã lỗi **547** (vi phạm khóa ngoại), trả về 400 với thông báo "Dữ liệu liên quan không tồn tại".

Bài 9 — Đăng ký, đăng nhập và bảo vệ API bằng JWT

9.1 Ý tưởng

1. **Đăng ký:** mật khẩu được **băm** (biến thành chuỗi không thể dịch ngược) rồi mới lưu vào SQL Server.
2. **Đăng nhập:** đúng email và mật khẩu thì server trả về một **access token (JWT)**, giống như một "vé vào cửa" có hạn sử dụng.
3. **Gọi API khác:** client gửi kèm vé trong header `Authorization: Bearer <token>`. Một **guard** ("người gác cổng") kiểm tra vé trước khi cho vào controller.

```
npm install @nestjs/jwt argon2
```

Argon2 là thuật toán băm mật khẩu được khuyến nghị hiện nay. **Không bao giờ** lưu mật khẩu gốc, và không dùng MD5/SHA để băm mật khẩu.

9.2 `UsersService`

```
// src/users/users.service.ts
@Injectable()
export class UsersService {
  constructor(@InjectRepository(User) private readonly usersRepo:
Repository<User>) {}
```

```
  create(email: string, passwordHash: string) {
    const user = this.usersRepo.create({ email, passwordHash });
    return this.usersRepo.save(user);
  }

  findByEmail(email: string) {
    return this.usersRepo.findOneBy({ email });
  }
}

// src/users/users.module.ts
@Module({
  imports: [TypeOrmModule.forFeature([User])],
  providers: [UsersService],
  exports: [UsersService], // cho AuthModule dùng
})
export class UsersModule {}
```

9.3 DTO, service và controller đăng nhập

```
nest g module auth
nest g service auth
nest g controller auth
```

```
// src/auth/dto/login.dto.ts
export class LoginDto {
  @IsEmail({}, { message: 'Email không hợp lệ' })
  email: string;

  @IsString()
  @MinLength(8, { message: 'Mật khẩu phải có ít nhất 8 ký tự' })
  password: string;
}
```

```
// src/auth/current-user.decorator.ts — thông tin chứa trong token
export interface JwtPayload {
  sub: string;   // id người dùng
  email: string;
}

// Lấy người dùng đang đăng nhập: myMethod(@CurrentUser() user: JwtPayload)
export const CurrentUser = createParamDecorator(
  (_data: unknown, ctx: ExecutionContext): JwtPayload =>
ctx.switchToHttp().getRequest().user,
);
```

```
// src/auth/auth.service.ts
import argon2 from 'argon2';

@Injectable()
export class AuthService {
  constructor(
    private readonly usersService: UsersService,
    private readonly jwtService: JwtService,
  ) {}

  async register(dto: LoginDto) {
    const passwordHash = await argon2.hash(dto.password); // băm mật khẩu
    return this.usersService.create(dto.email.toLowerCase(), passwordHash);
    // email trùng → SQL Server báo lỗi 2627 → SqlExceptionFilter trả 409
(Bài 8)
  }

  async login(dto: LoginDto) {
    const user = await
this.usersService.findByEmail(dto.email.toLowerCase());
    const isValid = user ? await argon2.verify(user.passwordHash,
dto.password) : false;
    if (!user || !isValid) {
      throw new UnauthorizedException('Email hoặc mật khẩu không đúng'); //
không nói rõ sai cái nào
    }
    const payload: JwtPayload = { sub: user.id, email: user.email };
    return { accessToken: await this.jwtService.signAsync(payload) };
  }
}
```

```
// src/auth/auth.module.ts
@Module({
  imports: [
    UsersModule,
    JwtModule.registerAsync({
      global: true,                // JwtService dùng được ở mọi module
      inject: [ConfigService],
      useFactory: (config: ConfigService) => ({
        secret: config.getOrThrow<string>('JWT_SECRET'), // khóa bí mật để ký
token (lấy từ .env)
        signOptions: { expiresIn: '1h' },                // token hết hạn
sau 1 giờ
      }),
    }),
  ],
  controllers: [AuthController],
  providers: [AuthService],
})
export class AuthModule {}
```

9.4 Guard — người gác cổng

```
// src/auth/public.decorator.ts
export const IS_PUBLIC_KEY = 'isPublic';
export const Public = () => SetMetadata(IS_PUBLIC_KEY, true); // đánh dấu
route không cần đăng nhập
```

```
// src/auth/auth.guard.ts
@Injectable()
export class AuthGuard implements CanActivate {
  constructor(
    private readonly jwtService: JwtService,
    private readonly reflector: Reflector,
  ) {}

  async canActivate(context: ExecutionContext): Promise<boolean> {
    // 1. Route có @Public() thì cho qua
    const isPublic = this.reflector.getAllAndOverride<boolean>(IS_PUBLIC_KEY,
[
      context.getHandler(),
      context.getClass(),
```

```
    ]);
    if (isPublic) return true;

    // 2. Lấy token từ header "Authorization: Bearer <token>"
    const request = context.switchToHttp().getRequest();
    const [type, token] = request.headers.authorization?.split(' ') ?? [];
    if (type !== 'Bearer' || !token) throw new UnauthorizedException('Bạn
chưa đăng nhập');

    // 3. Kiểm tra token, gắn thông tin người dùng vào request
    try {
      request.user = await this.jwtService.verifyAsync<JwtPayload>(token);
    } catch {
      throw new UnauthorizedException('Phiên đăng nhập không hợp lệ hoặc đã
hết hạn');
    }
    return true;
  }
}
```

Đăng ký guard cho **toàn bộ ứng dụng** trong `AppModule`. Từ giờ mọi API đều cần đăng nhập, trừ route có `@Public()`. Cách này an toàn hơn gắn guard từng chỗ, vì quên một chỗ là lộ dữ liệu.

```
// src/app.module.ts
providers: [
  { provide: APP_GUARD, useClass: AuthGuard },
  { provide: APP_INTERCEPTOR, useClass: ClassSerializerInterceptor },
  { provide: APP_FILTER, useClass: SqlExceptionFilter },
],
```

```
// src/auth/auth.controller.ts
@Controller('auth')
export class AuthController {
  constructor(private readonly authService: AuthService) {}

  @Public()
  @Post('register')
  register(@Body() dto: LoginDto) {
    return this.authService.register(dto); // trả user, passwordHash đã bị ẩn
  }
```

```
  @Public()
  @Post('login')
  @HttpCode(200)
  login(@Body() dto: LoginDto) {
    return this.authService.login(dto);
  }

  @Get('me') // không có @Public() → cần token
  me(@CurrentUser() user: JwtPayload) {
    return user;
  }
}
```

9.5 Mỗi người chỉ thấy công việc của mình

```
// src/tasks/tasks.service.ts — mọi hàm nhận thêm userId
findAll(userId: string) {
  return this.tasksRepo.find({ where: { userId }, order: { createdAt: 'DESC'
} });
}

async findOne(userId: string, id: string) {
  const task = await this.tasksRepo.findOneBy({ id, userId }); // phải khớp
CẢ id VÀ userId
  if (!task) throw new NotFoundException('Không tìm thấy công việc');
  return task;
}

create(userId: string, dto: CreateTaskDto) {
  const task = this.tasksRepo.create({ ...dto, userId });
  return this.tasksRepo.save(task);
}
// update và remove gọi this.findOne(userId, id) trước, nên tự động được bảo
vệ
```

```
// src/tasks/tasks.controller.ts
@Get()
findAll(@CurrentUser() user: JwtPayload) {
  return this.tasksService.findAll(user.sub);
}

@Post()
create(@CurrentUser() user: JwtPayload, @Body() dto: CreateTaskDto) {
  return this.tasksService.create(user.sub, dto);
}
```

Điều kiện `userId` nằm **ngay trong câu truy vấn** là cực kỳ quan trọng. Thiếu nó, người dùng A chỉ cần đổi id trên URL là xem và sửa được việc của người dùng B. Đây là một trong những lỗ hổng bảo mật phổ biến nhất trên thực tế.

9.6 Thử toàn bộ luồng

1. `POST /auth/register` với `{"email":"an@example.com","password":"matkhau123"}` → trả user (không có `passwordHash`).
2. Gọi lại lần nữa → **409** "Dữ liệu đã tồn tại".
3. `POST /auth/login` → nhận `accessToken`.
4. `GET /tasks` **không** kèm token → **401** "Bạn chưa đăng nhập".
5. `GET /tasks` kèm header `Authorization: Bearer <accessToken>` → danh sách công việc của bạn.

Tự luyện

1. Tạo hai tài khoản; dùng token của người thứ hai để `PATCH` một task của người thứ nhất. Kết quả phải là 404.
2. Đổi `expiresIn` thành `'1m'`, chờ một phút rồi gọi lại API để thấy token hết hạn.
3. Dán token vào trang **jwt.io** để xem nội dung bên trong. Bạn sẽ thấy: token **không được mã hóa**, chỉ được ký. Vì vậy không bao giờ đặt mật khẩu hay thông tin nhạy cảm vào token.

Bài 10 — Thử API bằng Swagger và viết unit test đầu tiên

10.1 Swagger: trang tài liệu và thử API ngay trên trình duyệt

```
npm install @nestjs/swagger
```

```
// src/main.ts — phiên bản hoàn chỉnh
import { ValidationPipe } from '@nestjs/common';
import { NestFactory } from '@nestjs/core';
import { DocumentBuilder, SwaggerModule } from '@nestjs/swagger';
import { AppModule } from './app.module.js';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  app.useGlobalPipes(new ValidationPipe({ whitelist: true,
forbidNonWhitelisted: true, transform: true }));

  const config = new DocumentBuilder()
    .setTitle('Task API')
    .addBearerAuth() // thêm nút "Authorize" để dán token
    .build();
  SwaggerModule.setup('docs', app, () => SwaggerModule.createDocument(app,
config));

  await app.listen(process.env.PORT ?? 3000);
}
await bootstrap();
```

Mô tả DTO cho Swagger bằng `@ApiProperty` (kèm ví dụ), và nhóm API bằng `@ApiTags`:

```
// src/tasks/dto/create-task.dto.ts
import { ApiProperty } from '@nestjs/swagger';

export class CreateTaskDto {
  @ApiProperty({ example: 'Học NestJS bài 7' })
  @IsString()
  @IsNotEmpty({ message: 'Tiêu đề không được để trống' })
  @MaxLength(200, { message: 'Tiêu đề tối đa 200 ký tự' })
  title: string;
}
```

```
// src/tasks/dto/update-task.dto.ts — đổi PartialType sang bản của
@nestjs/swagger
import { ApiPropertyOptional, PartialType } from '@nestjs/swagger';

export class UpdateTaskDto extends PartialType(CreateTaskDto) {
  @ApiPropertyOptional({ example: true })
  @IsOptional()
  @IsBoolean({ message: 'done phải là true hoặc false' })
  done?: boolean;
}

// src/tasks/tasks.controller.ts
@ApiTags('tasks')      // nhóm "tasks" trên trang Swagger
@ApiBearerAuth()       // các API này cần token
@Controller('tasks')
export class TasksController { ... }
```

Cách dùng trang Swagger:

1. Mở `http://localhost:3000/docs`.
2. Mở mục `POST /auth/login`, bấm **Try it out**, nhập email và mật khẩu, bấm **Execute**. Copy `accessToken` trong kết quả.
3. Bấm nút **Authorize** ở đầu trang, dán token, bấm **Authorize**.
4. Thử các API của `tasks`: Swagger tự gửi kèm token.

Trang `/docs` cũng chính là tài liệu để bạn gửi cho người làm frontend hoặc mobile.

10.2 Unit test là gì?

Unit test là đoạn code tự động kiểm tra một phần nhỏ (thường là một service) có chạy đúng không. Nhờ đó, mỗi lần sửa code bạn chỉ cần chạy `npm test` để biết mình có làm hỏng gì không, thay vì thử bằng tay từng API.

Điểm hay của NestJS: nhờ dependency injection, ta thay repository thật bằng một **repository giả**. Test chạy được mà **không cần SQL Server**.

```
// src/tasks/tasks.service.spec.ts
import { NotFoundException } from '@nestjs/common';
import { Test } from '@nestjs/testing';
import { getRepositoryToken } from '@nestjs/typeorm';
import { Task } from './task.entity.js';
import { TasksService } from './tasks.service.js';
```

```
describe('TasksService', () => {
  let service: TasksService;

  // Repository GIẢ: không cần SQL Server để chạy test
  const fakeRepo = {
    findOneBy: vi.fn(),
    create: vi.fn((data) => data),
    save: vi.fn(async (task) => ({ id: 'task-1', ...task })),
  };

  beforeEach(async () => {
    vi.clearAllMocks(); // xóa lịch sử gọi hàm giả trước mỗi test
    const moduleRef = await Test.createTestingModule({
      providers: [
        TasksService,
        { provide: getRepositoryToken(Task), useValue: fakeRepo }, // dùng
repo giả thay repo thật
      ],
    }).compile();
    service = moduleRef.get(TasksService);
  });

  it('tạo công việc gắn với đúng người dùng', async () => {
    const task = await service.create('user-1', { title: 'Học NestJS' });
    expect(task).toEqual({ id: 'task-1', title: 'Học NestJS', userId: 'user-
1' });
  });

  it('báo lỗi 404 khi không tìm thấy công việc', async () => {
    fakeRepo.findOneBy.mockResolvedValue(null); // giả lập: database không có
task này
    await expect(service.findOne('user-1',
'abc')).rejects.toThrow(NotFoundException);
  });
});
```

```
npm test
# ✓ src/tasks/tasks.service.spec.ts (2 tests)
#   Tests  2 passed (2)
```

Kết quả trên là kết quả chạy thật khi soạn tài liệu.

Hàm	Ý nghĩa
`describe('...', () => {})`	Nhóm các test của một đối tượng
`it('...', async () => {})`	Một trường hợp test, mô tả bằng câu dễ hiểu
`beforeEach`	Chạy trước mỗi test (chuẩn bị)
`vi.fn()`	Tạo hàm giả
`mockResolvedValue(x)`	Cho hàm giả trả về `x` (bất đồng bộ)
`expect(a).toEqual(b)`	Kiểm tra `a` bằng `b`
`rejects.toThrow(X)`	Kiểm tra hàm async ném lỗi loại `X`

NestJS 12 dùng **Vitest** để chạy test. Video hướng dẫn cũ dùng Jest thì sẽ thấy `jest.fn()` thay cho `vi.fn()`, còn lại gần như giống hệt.

Tự luyện

1. Thêm test: `update` đổi đúng trường `done`.
2. Thêm test cho `AuthService.login`: sai mật khẩu thì ném `UnauthorizedException`. Gợi ý: tạo `UsersService` giả và `JwtService` giả bằng `{ provide: UsersService, useValue: {...} }`.
3. Cố tình sửa sai `TasksService.create` (bỏ `userId`) rồi chạy `npm test` để thấy test báo lỗi.

Tổng kết và bước tiếp theo

Hoàn thành 10 bài, bạn đã có một API thật: đăng ký, đăng nhập, mỗi người quản lý công việc của riêng mình, dữ liệu lưu trong SQL Server, có trang Swagger và unit test.

Cấu trúc dự án hoàn chỉnh

```
src/
├── main.ts                      ← ValidationPipe + Swagger
├── app.module.ts                ← kết nối SQL Server, guard, interceptor,
filter toàn cục
├── common/
|   └── sql-exception.filter.ts ← đổi lỗi SQL Server thành thông báo dễ hiểu
```

```
├── users/
│   ├── user.entity.ts
│   ├── users.service.ts
│   └── users.module.ts
├── auth/
│   ├── dto/login.dto.ts
│   ├── public.decorator.ts
│   ├── current-user.decorator.ts
│   ├── auth.guard.ts
│   ├── auth.service.ts
│   ├── auth.controller.ts
│   └── auth.module.ts
└── tasks/
    ├── dto/create-task.dto.ts
    ├── dto/update-task.dto.ts
    ├── task.entity.ts
    ├── tasks.service.ts
    ├── tasks.service.spec.ts
    ├── tasks.controller.ts
    └── tasks.module.ts
```

Mã nguồn đầy đủ của dự án đi kèm tài liệu này.

Đường đi của một request

Khi gọi `POST /tasks` kèm token, request đi qua các bước:

1. **AuthGuard** kiểm tra token. Sai hoặc thiếu → 401.
2. **ValidationPipe** kiểm tra body theo `CreateTaskDto`. Sai → 400.
3. **TasksController** nhận dữ liệu đã sạch, lấy người dùng bằng `@CurrentUser()`.
4. **TasksService** xử lý nghiệp vụ, gọi **Repository**.
5. **TypeORM** sinh câu T-SQL và gửi tới **SQL Server**.
6. Kết quả đi ngược ra; **ClassSerializerInterceptor** ẩn trường nhạy cảm.

Nếu có lỗi database ở bước 5, **SqlExceptionFilter** đổi nó thành thông báo dễ hiểu.

10 lỗi người mới hay gặp

Lỗi	Cách sửa
`Cannot find module './tasks.service'`	Thiếu đuôi `.js` khi import: `'./tasks.service.js'`
`Nest can't resolve dependencies of ...`	Thiếu `providers`, `exports` hoặc `imports` (Bài 4.5)
Không kết nối được SQL Server	Bật TCP/IP, Mixed Mode, kiểm tra cổng 1433 và file `.env` (Bài 7.7)
Chữ tiếng Việt thành dấu `?`	Dùng `NVARCHAR` và tiền tố `N'...'` trong SQL
Validation không chạy	Quên `useGlobalPipes`, hoặc DTO viết bằng `interface`
`GET /tasks/count` trả lỗi UUID	Route `:id` đứng trước route `count` — đặt route cụ thể lên trên
Mật khẩu lộ ra trong API	Thiếu `@Exclude()` hoặc `ClassSerializerInterceptor` (Bài 8.4)
Mọi API đều báo 401, kể cả đăng nhập	Quên `@Public()` ở route đăng ký/đăng nhập
User này sửa được việc của user khác	Thiếu điều kiện `userId` trong truy vấn (Bài 9.5)
Mất cột/dữ liệu sau khi sửa entity	Do `synchronize: true` — chỉ dùng ở máy học

Checklist tự đánh giá

- [ ] Giải thích được controller, service, module khác nhau thế nào
- [ ] Hiểu dependency injection và sửa được lỗi "can't resolve dependencies"
- [ ] Viết được DTO có kiểm tra dữ liệu
- [ ] Viết được câu lệnh T-SQL: tạo bảng, `SELECT` có `WHERE` / `JOIN`, `INSERT`, `UPDATE`, `DELETE`
- [ ] Kết nối NestJS với SQL Server và làm CRUD bằng TypeORM
- [ ] Làm được quan hệ một – nhiều giữa hai bảng
- [ ] Làm được đăng ký, đăng nhập, bảo vệ API bằng JWT
- [ ] Thử API bằng Swagger, viết được unit test cho service

Bài tập tổng hợp

Tự xây lại dự án từ đầu **mà không nhìn tài liệu**, rồi thêm:

1. Bảng `Projects`: một người có nhiều project, một project có nhiều task.
2. Trường `priority` (`low` / `medium` / `high`) và `dueDate` cho task.
3. API lọc: `GET /tasks?done=false&priority=high`.
4. Phân trang: `GET /tasks?page=2&limit=10`, dùng `take` và `skip` trong `find`. TypeORM sẽ sinh `OFFSET ... FETCH NEXT` như bạn đã học ở Bài 6.

Học tiếp gì sau tài liệu này?

Khi đã tự tin với các bài trên, chuyển sang **giáo trình NestJS nâng cao** (từ cơ bản tới thực chiến) đã soạn trước đó. Các chủ đề tiếp theo:

- Migration thay cho `synchronize`.
- Phân quyền theo vai trò (admin/user).
- Interceptor, middleware.
- Rate limit.
- E2E test.
- Hàng đợi công việc.
- Docker.

Giáo trình nâng cao dùng PostgreSQL. Khái niệm NestJS và TypeORM hoàn toàn giống nhau; chỉ khác phần cấu hình kết nối, kiểu dữ liệu (`nvarchar`, `bit`, `datetime2` thay cho `varchar`, `boolean`, `timestamptz`) và mã lỗi (2627 thay cho 23505).

Tài liệu chính thức nên đọc kèm:

- NestJS: docs.nestjs.com
- TypeORM: typeorm.io
- T-SQL: trang Microsoft Learn về SQL Server

Các địa chỉ này chưa được mở lại khi soạn tài liệu, bạn hãy tự kiểm tra.